

Keras 3.x Troubleshooting Guide

This guide covers the most common problems when setting up **Keras 3.x with the PyTorch backend** for Week 6, and how to fix them. It is the companion to the **Technical Setup Guide** — if you have not completed that yet, start there.

Before anything else, run these two checks. A large fraction of problems come from (1) the wrong environment being active, or (2) the backend not being set first.

```
# 1. Are you in the course environment? Prompt should show (ml_week6_env)
which python      # Windows: where python - path must contain "ml_week6_env"

# 2. Is the backend set before importing keras? Restart your kernel, then:
import os
os.environ['KERAS_BACKEND'] = 'torch'
import keras
print(keras.backend.backend())  # should print: torch
```

Issue 1 — “Cannot set backend after keras is imported”

Error message

RuntimeError: The backend cannot be changed once the package is imported.

Cause: you imported `keras` **before** setting `os.environ['KERAS_BACKEND'] = 'torch'`.

Solution

1. Restart your Python kernel/session (Jupyter: **Kernel** → **Restart**).
2. Set the backend **first**, then import:

```
import os
os.environ['KERAS_BACKEND'] = 'torch'
import keras  # now safe
```

Prevention: always put the two `os.environ` lines at the very top of every notebook/script, above all other imports. Alternatively set `KERAS_BACKEND=torch` permanently in your shell (see Setup Guide, Step 6).

Issue 2 — Keras 2.x installed instead of 3.x

Symptom: `keras.__version__` shows 2.x.x.

Cause: TensorFlow is installed in the same environment and ships its own bundled Keras 2.x, which shadows standalone Keras 3.

Solution

```
# Make sure the course environment is active first!
pip uninstall keras tensorflow      # remove old Keras and TensorFlow
pip install "keras>=3.0"           # reinstall standalone Keras 3
python -c "import keras; print(keras.__version__)"
```

Expected output: 3.x.x.

If you genuinely need TensorFlow for another project, keep it in a **separate** virtual environment. Mixing TensorFlow and standalone Keras 3 in one environment is the most common cause of this issue.

Issue 3 — PyTorch not detected

Error message

ImportError: PyTorch backend requires PyTorch to be installed.

Cause: PyTorch is not installed, or you are not in the environment where you installed it.

Solution

```
# Confirm you are in the course environment
which python      # Windows: where python - path must contain "ml_week6_env"

# Reinstall PyTorch (CPU example - use your hardware's command from the Setup Guide)
pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu

# Verify
python -c "import torch; print(torch.__version__)"
```

If it still fails: upgrade pip (`python -m pip install --upgrade pip`), close and reopen the terminal, re-activate the environment, and try again.

Issue 4 — GPU installed but not detected (`is_available()` returns False)

Symptom: you installed a CUDA or XPU wheel, but the verification prints `CUDA (NVIDIA): False` or `XPU (Intel): False`.

Causes and fixes

- **Driver too old or missing (most common).**
 - NVIDIA: run `nvidia-smi`. If it errors or shows a very old driver, update your NVIDIA driver, then reinstall the matching `cuXXX` wheel.
 - Intel: confirm the Intel GPU driver is installed (Setup Guide, Step 3, Option C).

- **You actually installed the CPU wheel.** Check the build tag:

```
import torch
print(torch.__version__)  # +cu128 / +xpu = GPU build; +cpu = CPU-only build
```

If it ends in `+cpu`, reinstall with the correct `--index-url` for your hardware.

- **Wrong CUDA version for your driver.** Try a lower `cuXXX` index (e.g. `cu126` instead of `cu130`).

This is not a blocker for Week 6. CPU training handles MNIST in well under a minute per epoch. If you cannot get the GPU recognized, switch to the CPU wheel and continue — your results will be identical, just slightly slower.

Issue 5 — MNIST download fails

Error message

URLError: <urlopen error ...>

or

Exception: URL fetch failure

Cause: network issues, a firewall/proxy blocking the download, or a corrupted local cache.

Solution A — download via torchvision instead

```

from torchvision import datasets
import os

os.makedirs('data', exist_ok=True)
mnist_train = datasets.MNIST(root='data', train=True, download=True)
mnist_test  = datasets.MNIST(root='data', train=False, download=True)

X_train = mnist_train.data.numpy()
y_train = mnist_train.targets.numpy()
X_test  = mnist_test.data.numpy()
y_test  = mnist_test.targets.numpy()

print(f"MNIST loaded: {X_train.shape[0]} training samples")

```

Solution B — clear a corrupted cache and retry

```

import os, shutil
cache_dir = os.path.expanduser('~/.keras/datasets')
if os.path.exists(cache_dir):
    shutil.rmtree(cache_dir)
    print("Cache cleared. Try loading MNIST again.")

```

Solution C — network/firewall

- Temporarily disable any VPN.
- Check corporate firewall/proxy settings.
- Try a different network (e.g. a mobile hotspot).

Issue 6 — Keras says “Using TensorFlow backend”

Symptom: `keras.backend.backend()` returns `'tensorflow'` instead of `'torch'`.

Cause: the backend was not set before importing keras.

Solution

```

# Restart the kernel first!
import os
os.environ['KERAS_BACKEND'] = 'torch'
print("Backend env var:", os.environ.get('KERAS_BACKEND'))    # torch

import keras
print("Keras is using:", keras.backend.backend())             # torch

```

Expected output

```

Backend env var: torch
Keras is using: torch

```

Issue 7 — “No module named ‘keras’”

Error message

```
ModuleNotFoundError: No module named 'keras'
```

Cause: Keras is not installed, or the wrong environment is active.

Solution

```
which python      # Windows: where python - must be the ml_week6_env path
pip install "keras>=3.0"
python -c "import keras; print('Keras installed')"
```

If it still fails: make sure the environment is activated, confirm `pip --version` and `python --version` point to the **same** environment, and try `python -m pip install "keras>=3.0"`.

Issue 8 — Training extremely slow on CPU

Symptom: training takes many minutes per epoch on MNIST.

Cause: an oversized model or too much data for a CPU.

Fixes

1. **Use a reasonably sized model** — the Week 6 baseline is fine:

```
# [OK] Good for CPU
model = keras.Sequential([
    keras.layers.Input(shape=(28, 28)),
    keras.layers.Flatten(),
    keras.layers.Dense(128, activation='relu'),
    keras.layers.Dense(64, activation='relu'),
    keras.layers.Dense(10, activation='softmax'),
])

# [WRONG] Overkill for CPU
# Dense(1024) -> Dense(512) -> Dense(256) -> Dense(10)
```

2. **Use fewer epochs** — 5 is plenty for the Week 6 demo:

```
history = model.fit(X_train, y_train, epochs=5, validation_split=0.2)
```

3. **Train on a subset while developing:**

```
X_small, y_small = X_train[:10000], y_train[:10000]
history = model.fit(X_small, y_small, epochs=5)
```

Expected CPU training time: roughly 30–60 seconds per epoch on the full 60,000-sample MNIST set.

Issue 9 — Jupyter kernel crashes during training

Symptom: the kernel dies or restarts during `model.fit()`.

Causes: insufficient RAM, a model that is too large, or too large a batch size.

Fixes

1. **Reduce the batch size:**

```
history = model.fit(X_train, y_train, epochs=5, batch_size=16)
```

2. **Use a smaller dataset:**

```
X_small, y_small = X_train[:20000], y_train[:20000]
history = model.fit(X_small, y_small, epochs=5)
```

3. **Restart Jupyter with a higher data-rate limit:**

```
# Stop Jupyter with Ctrl+C, then:
jupyter notebook --NotebookApp.iopub_data_rate_limit=1.0e10
```

General troubleshooting workflow

When something does not work, go through these in order:

1. **Restart the kernel** (Jupyter: **Kernel** → **Restart**).
2. **Confirm the backend is set first:**

```
import os
os.environ['KERAS_BACKEND'] = 'torch'
import keras
print(keras.backend.backend()) # should print 'torch'
```

3. **Check your versions:**

```
import keras, torch, numpy
print(f"Keras: {keras.__version__}")
print(f"PyTorch: {torch.__version__}")
print(f"NumPy: {numpy.__version__}")
```

4. **Re-run the full verification script** (Setup Guide, Step 7).
5. **If it is still broken, start clean:**
 - Deactivate and delete the environment folder (`ml_week6_env`).
 - Recreate it and reinstall following the Setup Guide, Steps 1–7.
6. **If it is STILL broken:**
 - Search the Keras issue tracker: <https://github.com/keras-team/keras/issues>
 - Ask on the course forum / Slack.
 - As a temporary fallback, use **Google Colab** for Week 6 (set the `KERAS_BACKEND=torch` backend there too).

Quick reference — symptom → section

What you see	Go to
“backend cannot be changed once imported”	Issue 1
<code>keras.__version__</code> is 2.x	Issue 2
“PyTorch backend requires PyTorch”	Issue 3
GPU wheel installed but <code>is_available()</code> is False	Issue 4
MNIST download / URL error	Issue 5
Backend reports <code>tensorflow</code>	Issue 6
“No module named ‘keras’”	Issue 7
Training is painfully slow	Issue 8
Kernel crashes during <code>fit()</code>	Issue 9
Anything else	General workflow